PGARA Demo Video Script

Oct 31, 2026 · @Samrender

Recording plan

The video runs about 7 minutes 30 seconds and shows only the PGARA website, no code, README or terminal. Each member speaks once, in one continuous block, and explains everything they built, and one driver shares the screen and does the actions in *italics* while the others speak.

Before recording

1. **Driver:** Shreyans shares one browser window for the whole video. Speakers don't share their own screens.
2. **Open before recording:** only the PGARA website, https://pgara.vercel.app.
3. **Settings used throughout:** Program **Rare error branch**, Registers **k = 4**, Training input **7 10**, Test input **13 10**, Live compile on. Zoom the browser to 110–125%.
4. **Google Meet:** everyone joins with camera on and stays muted except the speaker. Start Record from Activities with a VIT Workspace account. The MP4 lands in the organiser's Drive, under Meet Recordings.
5. **Read off the screen:** before recording, check that the numbers in parts 4 and 5 match what the Studio shows, and change the script if they don't.

Running order

#	Speaker	Explains	Time	Website tab
1	Nishanth	Who we are, the problem, the website	0:00–1:00	Home page
2	Shreyans	Reading the code, and finding hot and rare paths	1:00–2:20	Tokens, AST, TAC, CFG
3	Samrender	Checking the code, and the register auction	2:20–3:45	Symbols, Allocation
4	Karthikeya	Cleaning up the code, value of each variable, final code	3:45–5:05	SSA & Optimiser, Allocation, Target Code
5	Abhishek	Final fix-up, results, limits, closing	5:05–7:30	Allocation, Run & Metrics

Each block follows the author contributions in the report, so nobody has to come back for a second turn.

Script

1. Nishanth Kumar (0:00–1:00)

Screen: website home page.

"Hello everyone, we are Team 14 from VIT. I'm Nishanth, and with me are Shreyans, Samrender, Karthikeya and Abhishek. Our project is called PGARA.

A computer's processor has only a few registers. Registers are very fast, but memory is slow. So a compiler has to decide which values get a register and which go to memory.

Most compilers just guess which parts of a program run often. PGARA actually runs the program once, sees which parts are busy, and gives the registers to the values that need them most.

Why does this matter? A value used inside a loop may be read millions of times. If it sits in memory, every read is slow. If it sits in a register, every read is fast. So choosing well can make a program much faster.

We built the whole compiler ourselves, from reading the code to running it, and this website shows every step live.

(Point to the top bar.) On this website we type a program on the left. Here we choose the program and the number of registers. The tabs on the right show each step of the compiler. Shreyans will start."

2. Shreyans Chowdry (1:00–2:20)

Screen: Tokens, AST, TAC, then CFG tab.

"*(Tokens.)* First, the compiler reads the code and breaks it into small pieces called tokens, like words in a sentence. Each colour is a different kind of token.

(AST.) Next it builds this tree. The tree shows the structure of the program, for example which operation happens first.

(TAC.) Then the tree is turned into simple instructions, one small step per line.

(Grammar.) This tab shows the rules of our language. The tool checks that the rules are clear, so the compiler never gets confused about how to read a line.

(Back to the editor.) With Live compile on, everything updates as we type, so you can see each step change right away.

(CFG.) This graph shows every path the program can take. We run the program once and count how often each path is used. Red boxes run a lot, blue boxes run rarely. This error path runs only about 1% of the time, but a normal compiler would guess 50%. That difference is the key idea of PGARA. Over to Samrender."

3. Samrender Singh Saini (2:20–3:45)

Screen: Symbols tab, editor, then Allocation tab.

"*(Symbols.)* My first part is checking the program. This table lists every variable and where it can be used. If two variables have the same name in different places, we give them different names, like `i` and `i#1`, so they never get mixed up.

(Driver types a variable that doesn't exist.) If we make a mistake, the error shows up immediately with the line number. *(Undo.)* And now it's fixed.

We also catch other mistakes: calling a function with the wrong number of inputs, using an array like a normal variable, a break outside a loop, or a function that forgets to return a value.

(Allocation, the graph.) My second part is giving out the registers. In this graph, each circle is a value. A line between two circles means they are needed at the same time, so they can't share a register.

We visit the values in a special order. Because of the way our code is built, this order always works, so we never need the slow trial-and-error method that other compilers use.

(Scroll to the auction rounds.) We run it like an auction. Each value makes a bid. When there aren't enough registers, the highest bids win, shown in green, and the others go to memory, shown in red. Karthikeya will explain how each bid is decided."

4. Karthikeya Jami (3:45–5:05)

Screen: SSA & Optimiser, Allocation table, then Target Code.

"*(SSA & Optimiser.)* Before giving out registers, we clean up the code. We remove steps that are not needed and simplify calculations, so the program does less work.

We also rewrite the code so that every variable gets its value only once. This makes it much easier to track where each value comes from and where it is used.

(Allocation table.) Then we work out how valuable each value is. We ask one simple question: how many cycles do we save if this value stays in a register? That number is its bid. For example, these counters run all the time, so they are worth about 20,000 cycles. The error counters run rarely, so they are worth only about 180.

Working this out is very quick, only a few milliseconds per program. And values that work together in a loop can join hands and bid as a team, so they can share one register.

(Target Code.) Finally, we produce the machine code. Red lines are where a value had to be saved to memory. Over to Abhishek."

5. Abhishek Kumar (5:05–7:30)

Screen: Allocation tab, then Run & Metrics.

"*(Allocation.)* After the auction, we do one last check. If a value that lost can still find a free register, it takes it. We repeat this until nobody can do better. Then the result is stable.

(Run & Metrics.) Here we run the program on our simulator and count the cycles. On this program PGARA needs about 338,000 cycles, while the common method, linear scan, needs about 516,000. And the output is correct for every method.

(Point to the bar charts.) These charts compare four methods side by side: cycles, memory loads and stores, and code size. The best one is marked in green.

We also wrote 27 tests that check every step of the compiler, so we know the output is always correct.

We tested 17 programs and all 1,428 runs gave the correct answer. With four registers, PGARA was about 22% faster than linear scan, and about 3 to 4% faster than other good methods. It also moved values to memory half as often.

We also tried five different test inputs for each program, and the results stayed almost the same. So the improvement is not just luck.

There are some limits. PGARA helps most when registers are few, eight or fewer. And the test run must be similar to real use.

To sum up: PGARA measures instead of guessing, gives every value a fair price, and auctions the registers. Thank you for watching."

Rehearsal and questions

Do one full run-through before recording. If it goes over 8 minutes, shorten the front-end lines in part 2 and the results in part 5 first.

- [ ] Driver tests every click once: tab order, the error demo and the scroll to the auction rounds in part 3
- [ ] Numbers in parts 4 and 5 checked against the screen at k = 4 with inputs 7 10 / 13 10
- [ ] Each speaker ends with the hand-off line in their part

- [ ] Notifications off, browser zoom 110–125%, one tab per screen

Likely questions, by speaker

Speaker	Question	Short answer
Shreyans	Why LL(1) and recursive descent?	Predictable, one function per nonterminal, and the only conflict is the dangling else.
Shreyans	What is a coalition?	Values linked by φ-functions across a loop; they bid together so they can share one register and drop the copy.
Samrender	Why is semantic analysis L-attributed?	The scope environment flows top-down and left to right; expression types are built bottom-up.
Samrender	Why does chordality matter?	SSA interference graphs are chordal, so MCS gives a perfect elimination ordering and the auction needs no graph colouring.
Karthikeya	What are the two MDP states?	Register or memory; policy iteration gives the exact expected cycles saved, G(v).
Karthikeya	How fast is allocation?	1.2–18.9 ms per program; policy iteration averages 1.29 rounds.
Abhishek	What makes it a Nash equilibrium?	No spilled coalition can take a free register and improve on its own.
Abhishek	When does PGARA lose?	At k = 3 on matmul, Levenshtein and FIR, where Chaitin–Briggs packs tighter.
Nishanth	What if the profile is wrong?	Allocation is unchanged up to a 20% training error rate; beyond 30–40% the static prior is better.
Nishanth	Why is the gain zero at k ⩾ 16?	Nothing spills, so every auction-based allocator produces the same code.